

SkillAudit: Traceable Validation of Simplified Paper-Derived Agent Skill Specifications

Anonymous submission

Abstract

Large Language Model (LLM) agents can convert method descriptions in research papers into executable agent skill specifications, but simplifying these specifications may remove information required for method reproduction. Existing approaches to this task mainly focus on extracting method content and generating executable specifications, whereas methods for compressing prompts or skill specifications primarily optimize text length, execution cost, or downstream performance. However, these two lines of work rarely jointly record which source passages were removed, whether the remaining steps preserve all required dependencies, and which experimental results support the decision to accept or reject a simplified specification. In this paper, we present SkillAudit, a traceable post-compression validation protocol for candidate specifications produced by upstream simplification methods. It links procedural steps to their source passages and prerequisites, compares performance across three conditions (a no-specification baseline, the full specification, and a simplified candidate specification) on prespecified tasks, and produces an auditable record containing source provenance, the experimental configuration, test results, and the decision rationale. A compliant prospective audit also prespecifies its acceptance policy; because our six-state policy was specified after execution, we use it only for diagnostic replay and evaluate the matched evidence endpoints rather than claiming a prospectively validated admission policy. Across three fresh GPT-5.6 Sol repetitions on code-based method-reproduction tasks from 12 papers and 24 paper-task units, the full and simplified specifications improve operational success over the no-specification baseline by 0.264 and 0.266; their aggregate difference is 0.002 and varies locally. A known-truth suite recovers 40/40 deterministic labels, while exact-output-contract deletion yields 0/24 operational successes. The evidence supports task-local auditing, not candidate equivalence or universal admissibility.

Introduction

Large Language Model (LLM) agents can use natural-language instructions, external tools, and execution environments to complete complex multi-step tasks. As agent capabilities advance, method descriptions in research papers

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

are increasingly treated as reusable sources of procedural knowledge. Existing studies use agents to recreate code and experiments described in papers (Starace et al. 2025; Zhao et al. 2026), while paper-to-agent systems further transform papers and associated software into interactive agent artifacts (Miao et al. 2025). Through such upstream conversion processes, paper methods can be organized into executable agent skill specifications.

Existing research follows two related directions. The first extracts method content from papers or other knowledge sources and produces executable agent artifacts or reusable skills (Miao et al. 2025; Shen et al. 2026; Pan et al. 2026). The second reduces context length and execution cost through prompt compression or agent-skill compression (Jiang et al. 2023, 2024; Pan et al. 2024; Gao et al. 2026). For example, LLMingua removes tokens estimated to carry limited information, LongLLMingua introduces question-aware compression for long contexts, and LLMingua-2 formulates compression as token-level classification. However, removing tokens or sentences can disrupt semantic coherence and discard necessary information at high compression ratios (Pan et al. 2024; Liskavets et al. 2025).

A shorter skill specification does not necessarily retain all information required for method reproduction. Existing methods commonly evaluate simplification using compression ratio, execution cost, or downstream task performance, but rarely jointly record which source passages were removed, whether the remaining steps preserve their prerequisites, and which experimental results justify accepting or rejecting a candidate. Consequently, success on a small set of examples does not distinguish an insufficient full specification, a simplified candidate missing critical information, and an invalid experimental run. We therefore study candidate-level post-compression validation: given a full skill specification, a simplified candidate specification, and an executable evaluation package, determine whether the candidate can substitute for the full specification for a specified task under fixed experimental conditions.

We present SkillAudit, a traceable post-compression validation protocol. SkillAudit takes a source-to-step mapping confirmed by the evaluation-package builder, binds procedural steps to source passages and prerequisites, and checks whether a simplified candidate preserves the dependency closure required for task execution. Before held-out evaluation,

the protocol freezes the candidate, task, workspace, model, tools, run budget, tests, scorer, and prespecified acceptance criteria. It then executes three conditions: a no-specification baseline, the full specification, and the simplified candidate specification. Finally, SkillAudit combines held-out results with integrity checks, separates candidate rejection from baseline-, full-, and evidence-inconclusive states, and emits an auditable record containing source provenance, experimental configuration, results, and the decision rationale.

We evaluate matched evidence with GPT-5.6 Sol as the homogeneous primary executor. Three fresh schedule-matched repetitions yield 1,296 terminal rows from 12 papers and 24 paper-task units spanning NLP, software engineering, data analysis, and agent/tool use. Relative to the no-specification baseline, F and S improve operational success by 0.264 and 0.266, respectively, with positive paper-clustered intervals; their $S - F$ difference is 0.002 and its interval crosses zero. Both baseline contrasts remain positive in every repetition and domain, whereas the local $S - F$ ordering varies. A separate known-truth suite recovers all 40 deterministic labels and distinguishes identity or redundancy-preserving changes from deletion of a registered exact-output contract. Because the six-state policy was not bound before the primary executions, its pooled replay remains retrospective rather than confirmatory admission evidence. Together, the experiments support $B/F/S$ comparison and integrity-aware controls as task-local audit evidence; they do not establish candidate equivalence or universal admissibility.

Our contributions are threefold:

- We formulate candidate-level post-compression validation and distinguish failures caused by an insufficient full specification, missing information in a simplified candidate, baseline solvability, and invalid experimental runs.
- We introduce SkillAudit, which integrates source mapping, dependency-closure checking, controlled three-condition execution, a frozen evaluation boundary, and auditable decision records into a unified validation protocol.
- We provide matched evidence through three fresh GPT-5.6 Sol repetitions over 24 paper-task units and 4 domains, paper-clustered effects, known-truth controls, component ablation, and bounded robustness analyses, while exposing where the current registry remains inconclusive.

Related Work

From Papers to Executable Agent Artifacts

Recent systems turn scientific or external knowledge into executable research objects. PaperBench, AutoReproduce, RExBench, and REPRO-Bench evaluate code reconstruction, experiment reproduction with paper lineage, implementation of research extensions, and reproducibility assessment (Starace et al. 2025; Zhao et al. 2026; Edwards et al. 2026; Hu et al. 2025). Complementary construction systems organize source material as executable knowledge graphs, interactive paper agents, reusable toolsets, or skill libraries (Luo et al. 2026; Miao et al. 2025; Ma et al. 2025; Shen et al. 2026; Pan et al. 2026). These systems ask how to construct or reproduce

a useful artifact. SkillAudit begins after construction: it takes a fixed full artifact and a fixed simplified candidate as inputs, then evaluates a bounded substitution claim.

Context and Skill Compression

Prompt compression reduces model-facing context while attempting to preserve downstream behavior. LLMingua and LongLLMLingua use token-level and question-aware compression; LLMingua-2 reframes selection as token classification; later methods use sentence encoders, error-aware optimization, or training-free perception (Jiang et al. 2023, 2024; Pan et al. 2024; Liskavets et al. 2025; Larionov and Eger 2025; Tang et al. 2025). Agent-oriented work extends this objective to reusable instructions and skills (Gao et al. 2026; Xu and Wu 2026). In particular, SkillReducer compares no-skill, original, and compressed conditions, validates candidates in a real environment, and restores removed units after failures (Gao et al. 2026). SkillAudit does not claim these operations as new. Its narrower focus is a registry-mappable paper-derived candidate whose decision is bound to source locations, declared dependencies, a fixed task boundary, explicit invalid or inconclusive states, and a replayable evidence record.

Agent and Evaluator Validation

Tool-learning systems study tool construction, invocation, general-ability preservation, and efficient selection (Schick et al. 2023; Gao et al. 2024; Hao et al. 2025; Ma et al. 2025; Jia and Li 2026). AgentBoard, MCP-AgentBench, and Beyond Accuracy instead emphasize intermediate progress, protocol-aware interaction, evaluator validity, task difficulty, capability boundaries, and execution cost (Ma et al. 2024; Guo et al. 2026; Wang et al. 2026). Paper-reproduction benchmarks similarly freeze tasks or rubrics to distinguish system capability from evaluation artifacts (Starace et al. 2025; Zhao et al. 2026). These benchmarks produce scores, rankings, or capability profiles for complete agents. SkillAudit changes both the evaluation object and the output: it diagnoses whether B is nondiagnostic, whether F is sufficient, and whether S has matched evidence for local substitution, while retaining an explicit invalid state and source-linked record.

Problem Formulation

Let a paper P describe a bounded procedure that an LLM agent must reuse for a task t . An upstream extractor produces a *full specification* F , and an upstream reducer produces a shorter *candidate specification* S . Neither artifact is treated as an oracle: F may omit an operational requirement, while S may remove information needed by t . We ask a local substitution question under a fixed evaluation boundary Ω : can S replace F for (t, Ω) ? A no-specification baseline B completes the comparison set $C \in \{B, F, S\}$. It reveals tasks that the model can solve from the scaffold alone, whereas F tests whether the reference specification and evaluation package are sufficiently informative.

SkillAudit makes the candidate boundary inspectable by representing the paper procedure as source-addressed atoms $A = \{a_1, \dots, a_K\}$ and a directed dependency graph $G =$

(A, E) . Each atom is a typed, executable unit

$$a_i = (\text{id}_i, \ell_i, u_i, r_i, D_i), \quad (1)$$

where ℓ_i is an exact source locator, u_i is the model-facing instruction, r_i is its procedural role, and D_i contains direct prerequisites. A retained set $A_S \subsetneq A$ is structurally admissible only if it is nonempty, acyclic, and dependency closed. A deterministic renderer R then produces $F = R(A)$ and $S = R(A_S)$. This representation identifies what was retained and removed; it does not by itself establish behavioral sufficiency. The dependency edges are induced by the prerequisite sets, $E = \{(a_j, a_i) : a_j \in D_i\}$. Dependency closure therefore requires $a_i \in A_S \Rightarrow D_i \subseteq A_S$.

For registered block $r \in \{1, \dots, n\}$ and condition C , a fresh agent session yields a validity bit $v_{C,r}$, contract-pass bit $c_{C,r}$, patch-present bit $p_{C,r}$, and private score $q_{C,r} \in [0, 1]$. Here $c_{C,r} = 1$ means that every registered hard contract passed. Given a prespecified score threshold τ , operational success is

$$z_{C,r} = \mathbb{I}[v_{C,r} = 1 \wedge c_{C,r} = 1 \wedge p_{C,r} = 1 \wedge q_{C,r} \geq \tau], \quad (2)$$

$$k_C = \sum_{r=1}^n z_{C,r}.$$

Private cases evaluate one submitted patch and are not independent agent runs. Likewise, two tasks derived from one paper are related observations; our cross-paper inference therefore aggregates tasks within a paper and treats the paper as the resampling unit.

SkillAudit

SkillAudit is a post-compression validator rather than a reducer. It receives a fixed candidate, binds that candidate to its paper and task, executes matched $B/F/S$ blocks, validates the resulting evidence, and emits a bounded decision with a field-level rationale. Figure 1 separates upstream candidate construction from SkillAudit’s validation path. Passing source and closure checks establishes an inspectable, structurally well-formed boundary; it does not establish behavioral sufficiency. A compliant prospective audit seals the candidate, Ω , and γ before held-out execution begins. In this study, the $B/F/S$ executions are fresh, whereas the pooled six-state policy was constructed afterward and is therefore reported only as a retrospective diagnostic replay.

Source Binding and Candidate Construction

The evaluation-package builder registers atoms from quoted paper passages. Document digests and page, line, or byte spans identify the source; role labels distinguish operations, parameters, guards, ordering constraints, and output contracts. Dependencies are explicit because a prerequisite may occur in another paragraph, caption, or equation. Before rendering, the closure auditor rejects unknown identifiers, cycles, a non-strict candidate, and retained atoms whose prerequisites are absent. A fixed topological order with atom-identifier tie breaking makes repeated rendering byte stable.

Structural validity and behavioral sufficiency answer different questions. Keeping a solver instruction while dropping its preprocessing prerequisite violates closure before any model call. Dropping the solver together with its dependent steps can remain structurally valid while making the candidate behaviorally insufficient. SkillAudit preserves this distinction: the graph explains the candidate boundary, while held-out execution tests whether that boundary is adequate for the registered task.

Frozen Evaluation Boundary

The evaluation boundary contains every factor that can change the interpretation of a comparison:

$$\Omega = (P, A, G, B, F, S, t, W, M, T, U, Q, \sigma, \tau, \gamma), \quad (3)$$

where W is the starter workspace, M the model and decoding configuration, T the tool interface, U the action budget, Q the public tests, private cases, and scorer, σ the blocked condition schedule, and γ the decision policy. Cryptographic digests bind each concrete file to these symbols. Every condition begins from an isolated copy of W and a fresh conversation, and the registered schedule balances condition order and private-registry assignment.

Public tests provide visible development feedback, but the private scorer is terminal and one-way. Once the agent submits, its patch is frozen and scored without returning private results to the session. Case-generation seeds, registry assignment, schedule order, prompts, and model identifiers are recorded. When the executor does not expose an enforceable sampling seed, the evidence record distinguishes deterministic orchestration from nondeterministic model execution instead of claiming backend determinism.

Matched Execution and Evidence Validity

SkillAudit interprets validity before performance. A completed submission that violates a hard contract or scores below τ is a valid experimental failure and contributes $z_{C,r} = 0$ to the registered denominator. A row is invalid only when the registered execution cannot be interpreted, for example because a required digest or terminal output is absent or the scorer does not produce a registered result. Thus missing execution evidence cannot silently become evidence against S .

Let $V = 1$ only when every registered row and schedule-level integrity check is complete. Because F and S are matched within a block, SkillAudit retains the paired differences $d_r^z = z_{S,r} - z_{F,r}$ and $d_r^q = q_{S,r} - q_{F,r}$. Denote their interval bounds by $[L_z, U_z]$ and $[L_q, U_q]$. These paired quantities test candidate noninferiority; marginal totals alone cannot reveal whether F and S succeeded on different blocks.

Evidence-Aware Decision

The policy γ specifies a baseline ceiling $b_{\max}$, minimum full and candidate counts $f_{\min}$ and $s_{\min}$, and noninferiority margins ϵ_z and ϵ_q . Let $g_B = [k_B \leq b_{\max}]$, $g_F = [k_F \geq f_{\min}]$, and $g_S = [k_S \geq s_{\min}]$. Paired noninferiority is $N = [L_z \geq -\epsilon_z \wedge L_q \geq -\epsilon_q]$, while clear inferiority is $R =$

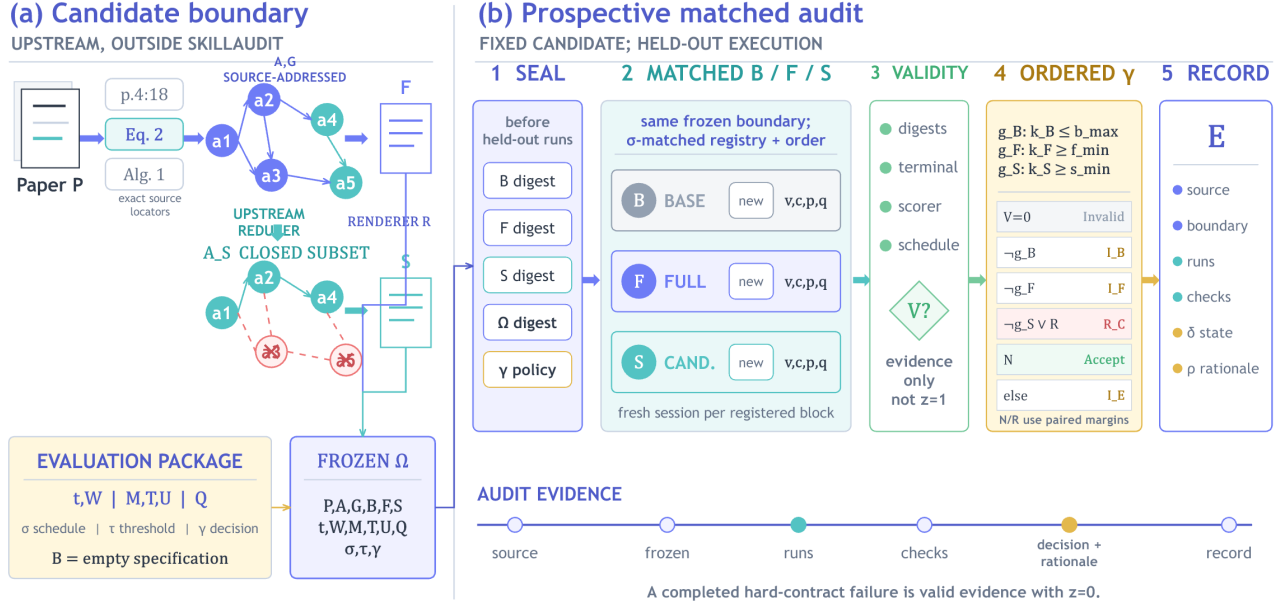


Figure 1: SkillAudit separates upstream candidate construction from validation. Source-addressed atoms and dependencies render F and a closed strict subset S ; the audit then seals $B/F/S$, Ω , and γ , executes matched blocks, validates terminal evidence, and binds the resulting diagnostic state to its source, boundary, checks, and rationale.

$[U_z < -\epsilon_z \vee U_q < -\epsilon_q]$. SkillAudit applies the ordered rule

$$\delta_{t,\Omega}(S) = \begin{cases} \text{Invalid,} & V = 0, \\ I_B, & V \wedge \neg g_B, \\ I_F, & V \wedge g_B \wedge \neg g_F, \\ R_C, & V \wedge g_B \wedge g_F \wedge (\neg g_S \vee R), \\ \text{Accept,} & V \wedge g_B \wedge g_F \wedge g_S \wedge N, \\ I_E, & \text{otherwise.} \end{cases} \quad (4)$$

Here I_B , I_F , R_C , and I_E denote baseline-inconclusive, full-inconclusive, reject-candidate, and evidence-inconclusive. The ordering matters: a strong baseline makes the task nondiagnostic; an insufficient F invalidates the reference comparison; and a wide paired interval withholds admission without asserting inferiority. Only **ACCEPT** authorizes substitution, and only for (t, Ω) .

The numerical policy is application-level: its thresholds, margins, interval construction, and minimum block count must be calibrated and sealed before an audit used for admission. Our primary experiment evaluates the matched endpoints rather than instantiate a confirmatory admission policy. For transparency, the supplementary material gives the exact values used in a separately labeled retrospective replay.

The supplementary material provides compact pseudocode for the ordered validation and recording procedure.

Auditable Evidence Record

The output is not only a label. SkillAudit emits

$$\mathcal{E} = (d_P, d_A, d_F, d_S, d_\Omega, \mathcal{O}, \mathcal{C}, \delta, \rho), \quad (5)$$

where the digest terms identify the paper, registry, rendered artifacts, and boundary; $\mathcal{O}$ contains the retained requests, trajectories, patches, and scores; $\mathcal{C}$ contains field-level integrity and decision checks; and ρ is a human-readable rationale. The record declares its retained evidence scope and cannot claim provenance beyond that scope. Accepted candidates can therefore be distributed with the exact task and evidence that authorize their use. Rejected, inconclusive, and invalid candidates retain a concrete repair path rather than an undifferentiated failure label.

Experiments

Questions and Design

The experiments separate utility, local substitutability, and audit validity. **RQ1** asks whether the full and simplified paper-derived specifications improve task outcomes over a no-specification baseline. **RQ2** asks whether S is distinguishable from F in aggregate and whether the answer changes across executions, papers, or domains. **RQ3** asks whether the registered decision logic and model-facing evaluation distinguish known-truth preserving and destructive transformations. Finally, offline component ablations and

bounded reducer, structural, and model comparisons probe which observations generalize beyond the primary construction. This organization follows evidence function rather than run chronology.

Cross-paper benchmark. The benchmark contains 12 papers, two tasks per paper, and four mechanism domains: NLP, software engineering, data analysis, and agent/tool use. The mechanisms are LLMingua, LLMingua-2, context-aware sentence compression, delta debugging, C-Reduce, Perses, HDBSCAN, Leiden, SnapATAC2, ReAct, Reflexion, and Toolformer (Jiang et al. 2023; Pan et al. 2024; Liskavets et al. 2025; Zeller and Hildebrandt 2002; Regehr et al. 2012; Sun et al. 2018; Campello et al. 2015; Traag, Waltman, and van Eck 2019; Zhang et al. 2024; Yao et al. 2023; Shinn et al. 2023; Schick et al. 2023). Each paper-task has a full specification, a dependency-closed strict-subset candidate, a starter workspace, public tests, two private registries, and hard output contracts. The registered primary family `S_dag_ratio_60_v1` applies the same deterministic DAG-ratio rule to every task: F contains 12 atoms and S contains 8, retaining 56.1–59.8% of specification tokens (58.0% pooled, a 42.0% reduction). The evaluation-package builder constructs the atoms, dependencies, tasks, and scorers from the cited methods; Section treats this shared construction process as a validity limitation.

Primary execution. We perform three fresh schedule-matched repetitions (SOL1–SOL3), each with six balanced $B/F/S$ blocks per paper-task. The homogeneous primary view contains 1,296 valid terminal rows: 24 tasks $\times$ 3 repetitions $\times$ 6 blocks $\times$ three conditions. Every response identifies the executor as `gpt-5.6-sol`; requests use temperature 0, top- p 1.0, at most 8,192 output tokens, and the registered private-score threshold $\tau = 0.99$. Each condition receives a fresh workspace and conversation, private feedback is terminal, and the registered schedules fix condition order and registry assignment. Because the executor exposes no enforceable sampling seed, the three repetitions are independent executions rather than controlled random-seed trials.

Endpoints and statistical units. The binary endpoint is operational success from Equation 2; the continuous endpoint is private score. Contrasts are paired within registered blocks. For cross-paper summaries, we aggregate both tasks and all repetitions within each paper, then draw 10,000 fixed-seed percentile-bootstrap samples over the 12 papers. These intervals describe sensitivity within the registered benchmark, not uncertainty over an unlimited population of papers. Private cases jointly score one patch and are not independent observations. A completed response that fails a hard contract remains a valid zero; a completed response alone is not task success.

Controls and auxiliary suites. Controls-v2 crosses four tasks, two registries, four arms, and three independent executions, yielding 96 valid model rows plus 40 deterministic known-truth fixtures. The four model arms are the full reference, an identity rendering, removal of registered redundancy, and removal of an exact-output contract. Offline component replay removes individual deterministic gates without

new model calls. Separate four-anchor suites compare alternate reducers, a three-level structural ladder, and six model slots. Because those suites reuse paper-task observations, differ in repetition depth, or alter candidate content, we report them as descriptive robustness evidence rather than independent causal effects or a model leaderboard.

RQ1: Paper-Derived Specifications Improve Outcomes

Table 1 reports the homogeneous Sol experiment. Across 432 rows per condition, B , F , and S achieve 81, 195, and 196 operational successes, with mean private scores 0.313, 0.609, and 0.608. Relative to B , F improves operational success by 0.264 and private score by 0.295; S improves the endpoints by 0.266 and 0.295. All four paper-clustered intervals are positive.

The direction is not produced by one repetition. SOL1 gives $B/F/S = 26/67/67$, SOL2 gives 26/65/66, and SOL3 gives 29/63/63 out of 144 rows per condition. Absolute trajectories vary, but F and S exceed B in every repetition. RQ1 is therefore supported within the registered paper set: paper-derived procedural information improves both endpoints on average.

RQ2: Aggregate Similarity Masks Local Heterogeneity

The $S - F$ contrast is much smaller than either baseline contrast. Its operational-success effect is 0.002 [-0.039, 0.049], and its private-score effect is -0.0004 [-0.027, 0.031]. Both intervals cross zero. The pooled analysis therefore detects no aggregate difference between S and F ; it does not establish superiority, equivalence, or task-level admissibility.

Figure 2(a) shows paper-level effects ranging from negative values to gains above 0.8. The largest improvements occur for Perses, C-Reduce, Reflexion, LLMingua, ReAct, and Leiden, whereas HDBSCAN remains difficult even with paper-derived information. At the domain level, $F - B$ and $S - B$ are positive for both endpoints in all four domains. Software engineering has the largest effects; data analysis has the smallest operational gains despite positive score gains. The $S - F$ direction is positive in NLP and software engineering but negative in data analysis and agent/tool use. RQ2 therefore has a bounded answer: the aggregate analysis does not resolve a difference between S and F , while their ordering remains paper- and domain-dependent.

RQ3: Known-Truth Controls Validate Bounded Audit Behavior

The deterministic layer recovers all 40 registered labels: 24 Admit, 8 Reject, and 8 Invalid. Table 2 then tests whether model-facing execution respects the intended transformation direction. The identity and redundancy arms remain close to F in aggregate, whereas deleting the exact-output contract produces zero operational successes. Among the 13 matched pairs in which F succeeds, every destructive counterpart fails (exact two-sided McNemar $p = 0.000244$).

The controls also expose the boundary of this result. Identity has three discordant pairs, so representation-preserving

Panel	Condition or contrast	Rows / papers	Successes	Operational success or effect	Private score or effect
Absolute	B	432 rows	81	0.188	0.313
	F	432 rows	195	0.451	0.609
	S	432 rows	196	0.454	0.608
Paired effect	$F - B$	12 papers	–	0.264 [0.116, 0.412]	0.295 [0.175, 0.429]
	$S - B$	12 papers	–	0.266 [0.120, 0.419]	0.295 [0.158, 0.448]
	$S - F$	12 papers	–	0.002 [-0.039, 0.049]	-0.0004 [-0.027, 0.031]

Table 1: Primary GPT-5.6 Sol results over 12 papers, 24 paper-tasks, and three fresh repetitions. Absolute rows use the frozen scorers over 432 terminal outputs per condition. Effect rows aggregate both tasks within each paper; brackets are 95% percentile paper-bootstrap sensitivity intervals. Both paper-derived artifacts improve over B , while $S - F$ remains near zero. These effects are not task-level admission rates.

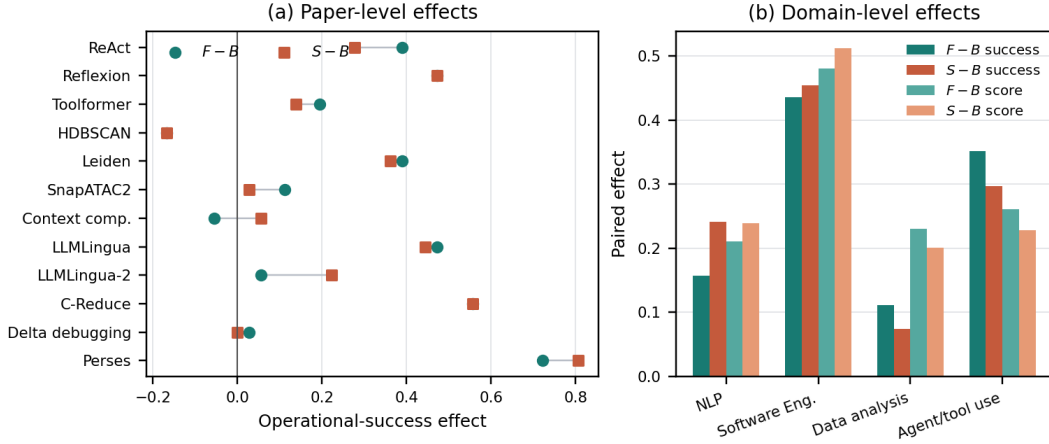


Figure 2: GPT-5.6 Sol effect heterogeneity. (a) Paper-level $F - B$ (circles) and $S - B$ (squares), aggregating two tasks and three repetitions per paper. (b) Domain means for operational-success and private-score effects. Zero denotes no aggregate effect; both artifact-versus-baseline effects are positive in every domain. The panels do not encode task-level admission.

Layer / arm	Correct or success	F only	Arm only
Full deterministic gate	40/40	–	–
Binary contract gate	32/40	–	–
Digest-only gate	32/40	–	–
F reference	13/24	–	–
Identity	14/24	1	2
Redundancy delete	12/24	1	0
Exact-contract delete	0/24	13	0

Table 2: Known-truth and model-facing controls. Deterministic rows report exact-state correctness; model rows report operational success over matched executions. “F only” and “Arm only” expose discordant pairs. The destructive arm validates sensitivity to one registered contract family, not population-wide calibration.

changes do not make stochastic trajectories row-wise invariant; redundancy deletion changes one matched outcome and supports no directional benefit. Terminal outcomes agree across all three executions for 28/32 task–registry–arm units, and the fluctuating units remain in the denominator. RQ3 supports deterministic label mechanics and sensitivity to

one destructive contract family, but not broad false-admit or false-reject rates.

Component and Generality Checks

The retrospective six-state replay admits no natural task: among 24 paper-task units it yields 0 ACCEPT, 7 baseline-inconclusive, 14 full-inconclusive, and 3 evidence-inconclusive states, with no reject-candidate or invalid states. This distribution exposes task and package limitations: seven scaffolds are already too capable, 14 full references miss the diagnostic minimum, and three paired intervals remain unresolved. It is therefore diagnostic evidence about where the current registry cannot support substitution, not validation of admission accuracy.

Offline ablation shows why the audit is more than a marginal F/S comparison. Removing B changes 7/24 retrospective task states. On the known-truth fixtures, binary contract-only and digest-only gates each recover 32/40 states, whereas contract tri-state and the full integrity-aware gate recover 40/40. Thus the baseline changes diagnosis, and the distinction between failed and missing contract evidence is necessary on the registered fixture suite. Because the natural tasks lack known admission truth and the pooled policy is

retrospective, these are component-sensitivity results rather than calibrated accuracy estimates.

The supplementary checks reinforce this cautious interpretation. On four Sol anchors, alternate reducers each succeed in 44/72 rows versus 79/144 separately paired F rows, while ladder success changes from 40/72 and 40/72 to 35/72. Across 6 model slots, $F - B$ and $S - B$ remain positive but $S - F$ changes sign. On the four shared anchors, Sol S uses 3.4% fewer executor-reported total tokens per request and has 47.7% lower median wall time than F . Shared anchors, fixed-scaffold tokens, and unequal repetitions make these descriptive robustness and system-cost results, not causal reducer effects or a model leaderboard.

Discussion

Traceability and sufficiency are complementary. Source locators and dependency closure make the candidate boundary inspectable, but neither proves behavioral sufficiency. Conversely, a private score without a source map cannot explain what paper content was retained. SkillAudit connects these views by testing the mapped boundary through matched $B/F/S$ execution.

The baseline and controls answer different validity questions. The no-specification condition succeeds in 81/432 executions, and adding B changes 7/24 retrospective task states; B therefore identifies tasks whose scaffold is already sufficient. Known-truth fixtures instead validate the implemented decision logic, while the destructive arm checks sensitivity to one contract family. Neither result independently verifies the author-constructed atoms, tasks, or scorers, so we do not convert control behavior into population sensitivity or specificity.

Candidate quality is local, and inconclusive states preserve action. The aggregate Sol $S - F$ intervals cross zero, but paper, domain, reducer, and model-setting orderings vary. Aggregate similarity therefore cannot authorize every substitution. SkillAudit preserves the reason: baseline-inconclusive calls for a more diagnostic task, full-inconclusive for repair of F or the evaluation package, rejection for candidate revision, and evidence-inconclusive for stronger matched evidence.

Evidence records bound reuse. An accepted candidate is not a task-independent “core” of a paper; it substitutes for F only under one (t, Ω) . Reuse should therefore carry the paper digest, atom boundary, model, task, workspace, scorer, registry, schedule, thresholds, terminal ledger, and rationale. Changing any field creates a new claim that requires a new audit.

Limitations and Broader Impact

The benchmark contains 12 papers and two tasks per paper across four domains, but remains a finite, constructed registry. Paper-bootstrap intervals measure sensitivity within this registry, not uncertainty over all scientific papers. One evaluation-package builder defines atoms, dependencies, tasks, private tests, and F ; independent expert construction, blinded scorer review, and agreement measurement are

needed before treating the registry as general paper-fidelity ground truth.

The six-state policy is prospective in form, but its thresholds are not calibrated to population false-admit risk and its pooled replay was constructed after the Sol registration. Controls-v2 covers four tasks and one destructive contract family; component replay has no independent truth; reducer, ladder, and model suites share four anchors and differ in execution depth. These analyses support bounded sensitivity and robustness, not confirmatory admission, causal component effects, or general cross-model rankings.

The executor exposes no enforceable sampling seed. Deterministic orchestration and analysis seeds are recorded, and aggregate baseline contrasts persist across three fresh calls, but individual outcomes vary. External bundles change both model and execution configuration, and request-token counts include the fixed scaffold. The reported analyses are scoped to the retained 2,688-row terminal ledger; the artifact does not support attempt-level reconstruction beyond this declared scope.

No human participants were studied, so we make no claims about researcher comprehension or reproduction speed. SkillAudit may reduce overconfident reuse by exposing decision boundaries, but detaching ACCEPT from its paper-task and scorer can create false assurance. Paper-derived code also retains execution and supply-chain risks; releases should preserve inconclusive, rejected, and invalid records rather than publish only favorable candidates.

Conclusion

SkillAudit combines source-addressed atoms, dependency closure, matched no-specification/full/candidate execution, evidence validity, diagnostic states, and a replayable record for task-local substitution audits. Across three GPT-5.6 Sol repetitions, both paper-derived artifacts improve operational success and private score over B on 12 papers and 24 paper-task units, while $S - F$ remains near zero in aggregate and heterogeneous locally. Known-truth controls recover every deterministic label and detect removal of a registered exact-output contract. The evidence supports traceable local auditing, not task-independent equivalence, universal admissibility, or independently calibrated registry validity.

References

- Campello, R. J. G. B.; Moulavi, D.; Zimek, A.; and Sander, J. 2015. Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection. *ACM Transactions on Knowledge Discovery from Data*, 10(1): 5:1–5:51.
- Edwards, N.; Lee, Y.; Mao, Y. A.; Qin, Y.; Schuster, S.; and Kim, N. 2026. RExBench: Can Coding Agents Autonomously Implement AI Research Extensions? In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16380–16417.
- Gao, S.; Shi, Z.; Zhu, M.; Fang, B.; Xin, X.; Ren, P.; Chen, Z.; Ma, J.; and Ren, Z. 2024. Confucius: Iterative Tool Learning from Introspection Feedback by Easy-to-Difficult

- Curriculum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 18030–18038.
- Gao, Y.; Li, Z.; Yuan, Y.; Ji, Z.; Ma, P.; and Wang, S. 2026. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. arXiv:2603.29919.
- Guo, Z.; Xu, B.; Zhu, C.; Hong, W.; Wang, X.; and Mao, Z. 2026. MCP-AgentBench: Evaluating Real-World Language Agent Performance with MCP-Mediated Tools. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 30888–30896.
- Hao, Y.; Cao, P.; Jin, Z.; Liao, H.; Chen, Y.; Liu, K.; and Zhao, J. 2025. CITI: Enhancing Tool Utilizing Ability in Large Language Models Without Sacrificing General Performance. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 23996–24004.
- Hu, C.; Zhang, L.; Lim, Y.; Wadhwani, A.; Peters, A.; and Kang, D. 2025. REPRO-Bench: Can Agentic AI Systems Assess the Reproducibility of Social Science Research? In *Findings of the Association for Computational Linguistics: ACL 2025*, 23616–23626.
- Jia, J.; and Li, Q. 2026. AutoTool: Efficient Tool Selection for Large Language Model Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 31265–31273.
- Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 13358–13376.
- Jiang, H.; Wu, Q.; Luo, X.; Li, D.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2024. LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- Larionov, D.; and Eger, S. 2025. PromptOptMe: Error-Aware Prompt Compression for LLM-based MT Evaluation Metrics. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 11807–11820.
- Liskavets, B.; Ushakov, M.; Roy, S.; Klivanov, M.; Etemad, A.; and Luke, S. K. 2025. Prompt Compression with Context-Aware Sentence Encoding for Fast and Improved LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 24595–24604.
- Luo, Y.; Yu, Z.; Wang, X.; Zhu, Y.; Zhang, N.; Wei, L.; Du, L.; Zheng, D.; and Chen, H. 2026. What Makes AI Research Replicable? Executable Knowledge Graphs as Scientific Knowledge Representations. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 841–861.
- Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. arXiv:2401.13178.
- Ma, Z.; Huang, Z.; Liu, J.; Wang, M.; Zhao, H.; and Li, X. 2025. Automated Creation of Reusable and Diverse Toolsets for Enhancing LLM Reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 24821–24830.
- Miao, J.; Davis, J. R.; Zhang, Y.; Pritchard, J. K.; and Zou, J. 2025. Paper2Agent: Reimagining Research Papers as Interactive and Reliable AI Agents. arXiv:2509.06917.
- Pan, Q.; Yang, Y.; Li, J.; Zhou, J.; Chen, K.; Li, X.; Chen, Q.; and He, L. 2026. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents. arXiv:2606.09316.
- Pan, Z.; Wu, Q.; Jiang, H.; Xia, M.; Luo, X.; Zhang, J.; Lin, Q.; Ruhle, V.; Yang, Y.; Lin, C.-Y.; Zhao, H. V.; Qiu, L.; and Zhang, D. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Regehr, J.; Chen, Y.; Cuoq, P.; Eide, E.; Ellison, C.; and Yang, X. 2012. Test-Case Reduction for C Compiler Bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 335–346.
- Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36*.
- Shen, S.; Cheng, W.; Ma, M.; Turcan, A.; Zhang, M. J.; and Ma, J. 2026. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. arXiv:2604.03964.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36*.
- Starace, G.; Jaffe, O.; Sherburn, D.; Aung, J.; Chan, J. S.; Maksin, L.; Dias, R.; Mays, E.; Kinsella, B.; Thompson, W.; Heidecke, J.; Glaese, A.; and Patwardhan, T. 2025. PaperBench: Evaluating AI’s Ability to Replicate AI Research. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Sun, C.; Li, Y.; Zhang, Q.; Gu, T.; and Su, Z. 2018. Perses: Syntax-Guided Program Reduction. In *Proceedings of the 40th International Conference on Software Engineering*, 361–371.
- Tang, J.; Xu, J.; Lu, T.; Zhang, Z.; Zhao, Y.; Hai, L.; and Zheng, H.-T. 2025. Perception Compressor: A Training-Free Prompt Compression Framework in Long Context Scenarios. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 4093–4108.
- Traag, V. A.; Waltman, L.; and van Eck, N. J. 2019. From Louvain to Leiden: Guaranteeing Well-Connected Communities. *Scientific Reports*, 9: 5233.
- Wang, Q.; Hu, Y.; Lu, M.; Wu, J.; Liu, Y.; and Tang, Y. 2026. Beyond Accuracy: A Cognitive Load Framework for Mapping the Capability Boundaries of Tool-Use Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 33611–33619.

Xu, X.; and Wu, W. 2026. Compression, Structure, and Executor Capability: A Controlled Real-Cost Decomposition of Language-Model Agent Skill Optimisation. arXiv:2607.03048.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zeller, A.; and Hildebrandt, R. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering*, 28(2): 183–200.

Zhang, K.; Zemke, N. R.; Armand, E. J.; and Ren, B. 2024. A Fast, Scalable and Versatile Tool for Analysis of Single-Cell Omics Data. *Nature Methods*, 21: 217–227.

Zhao, X.; Sang, Z.; Li, Y.; Shi, Q.; Zhao, W.; Wang, S.; Zhang, D.; Han, X.; Liu, Z.; and Sun, M. 2026. AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21920–21942.